

DATA ANALYTICS SQL ASSIGNMENT 1 :

DDL COMMANDS & CONSTRAINTS

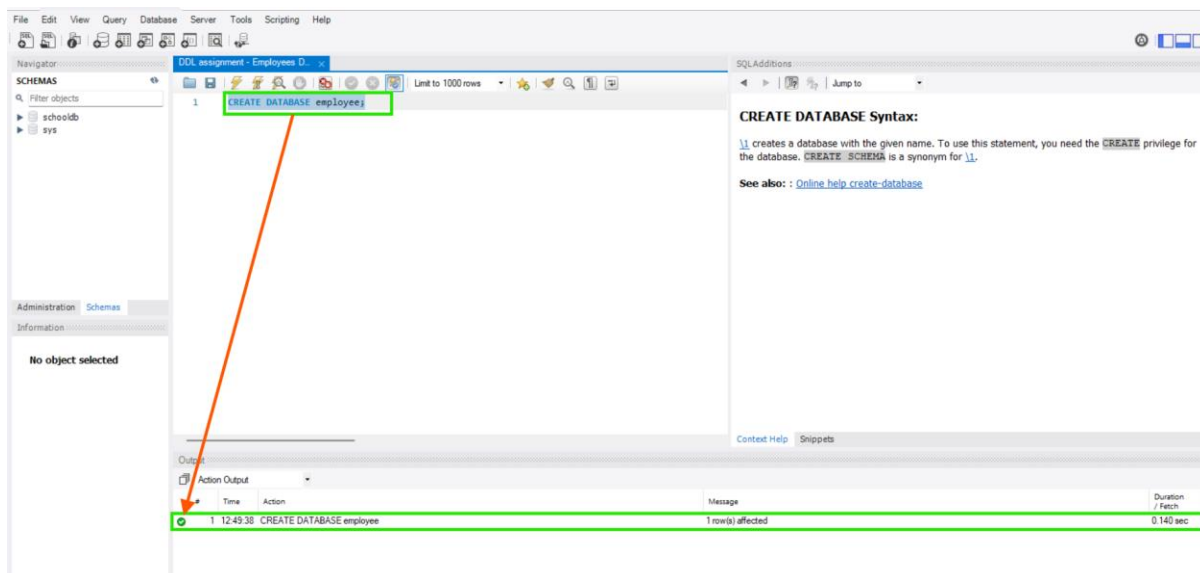
TOOLS : MYSQL / MYSQL WORKBENCH

PROJECT OBJECTIVE : DATABASE CREATION, TABLE ALTERATION,
CONSTRAINTS AND DATA INTEGRITY

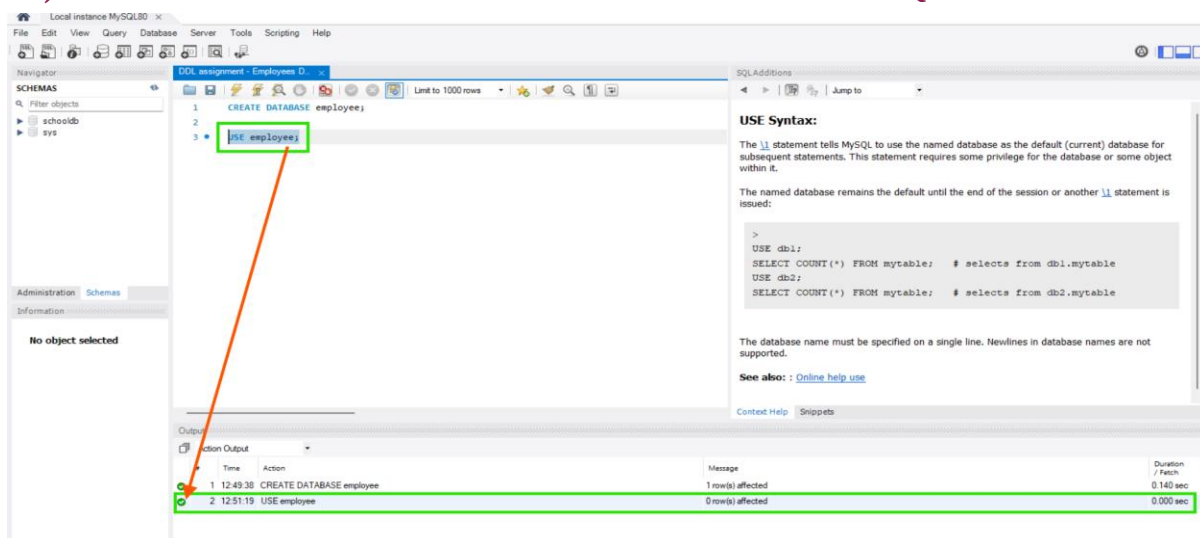
DATE OF SUBMISSION : 11/09/2026 – 1:30PM

1) DATABASE CREATION:

1.1) CREATE THE "EMPLOYEE" DATABASE.



1.2) SELECT THE "EMPLOYEE" DATABASE TO USE FOR SUBSEQUENT COMMANDS.



2) TABLE CREATION:

2.1) CREATE TABLE "DEPARTMENTS" WITH COLUMNS "DEPARTMENT_ID" AND "DEPARTMENT_NAME".

The screenshot shows the SQL Developer interface with the 'DDL assignment - Employees 0...' script open. The script contains the following SQL statements:

```
1 CREATE DATABASE employee;
2
3 USE employee;
4
5 CREATE TABLE Departments (
6   department_id INT,
7   department_name VARCHAR(100)
8 );
```

An orange arrow points from the 'CREATE TABLE Departments' statement in the script to the corresponding entry in the 'Action Output' table at the bottom. The 'Action Output' table shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	12:49:38	CREATE DATABASE employee	1 row(s) affected	0.140 sec
2	12:51:19	USE employee	0 row(s) affected	0.000 sec
3	12:52:51	CREATE TABLE Departments (department_id INT, department_name VARCHAR(100))	0 row(s) affected	0.031 sec

The right-hand pane displays the 'CREATE TABLE Syntax' documentation, explaining that the `CREATE` statement creates a table with the given name and that tables are created in the default database using the `InnoDB` storage engine.

2.2) CREATE TABLE "LOCATION" WITH COLUMNS "LOCATION_ID" AND "LOCATION".

The screenshot shows the SQL Developer interface with the 'DDL assignment - Employees 0...' script open. The script contains the following SQL statements:

```
1 CREATE DATABASE employee;
2
3 USE employee;
4
5 CREATE TABLE Departments (
6   department_id INT,
7   department_name VARCHAR(100)
8 );
9
10 CREATE TABLE Location (
11   location_id INT,
12   location VARCHAR(300)
13 );
```

An orange arrow points from the 'CREATE TABLE Location' statement in the script to the corresponding entry in the 'Action Output' table at the bottom. The 'Action Output' table shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	12:49:38	CREATE DATABASE employee	1 row(s) affected	0.140 sec
2	12:51:19	USE employee	0 row(s) affected	0.000 sec
3	12:52:51	CREATE TABLE Departments (department_id INT, department_name VARCHAR(100))	0 row(s) affected	0.031 sec
4	12:54:26	CREATE TABLE Location (location_id INT, location VARCHAR(300))	0 row(s) affected	0.015 sec

The right-hand pane displays the 'CREATE TABLE Syntax' documentation, explaining that the `CREATE` statement creates a table with the given name and that tables are created in the default database using the `InnoDB` storage engine.

2.3) CREATE TABLE "EMPLOYEES" WITH COLUMNS FOR ID, NAME, GENDER, AGE, HIRE DATE, DESIGNATION, DEPARTMENT, LOCATION AND SALARY.

CREATE TABLE Syntax:

[\i](#) creates a table with the given name. You must have the **CREATE** privilege for the table.

By default, tables are created in the default database, using the **InnoDB** storage engine. An error occurs if the table exists, if there is no default database, or if the database does not exist.

MySQL has no limit on the number of tables. The underlying file system may have a limit on the number of files that represent tables. Individual storage engines may impose engine-specific constraints. **InnoDB** permits up to 4 billion tables.

For information about the physical representation of a table, see [\i](#).

See also: [Online help create-table](#)

Output:

#	Time	Action	Message	Duration / Fetch
1	12:49:36	CREATE DATABASE employee	1 row(s) affected	0.140 sec
2	12:51:19	USE employee	0 row(s) affected	0.000 sec
3	12:52:51	CREATE TABLE Departments (department_id INT, department_name VARCHAR(100))	0 row(s) affected	0.031 sec
4	12:54:26	CREATE TABLE Location (location_id INT, location VARCHAR(100))	0 row(s) affected	0.015 sec
5	12:56:40	CREATE TABLE Employees (employee_id INT, employee_name VARCHAR(50), gender ENUM('M','F'), age INT, hire_date DATE, designation VARCHAR(100), department_id INT, location_id INT, salary DECIMAL(10,2))	0 row(s) affected	0.016 sec

3) ALTERING TABLE STRUCTURE:

3.1) ADD A NEW COLUMN "EMAIL" TO THE "EMPLOYEES" TABLE.

The screenshot shows a SQL IDE interface with a Navigator on the left, a central editor, and a right-hand pane for SQL documentation. The editor contains SQL code for creating and altering tables. The 'ALTER TABLE Employees ADD email VARCHAR(100);' statement is highlighted with a green box. The right-hand pane displays the 'ALTER TABLE Syntax' documentation. The bottom 'Output' pane shows a list of executed actions, with the final action 'ALTER TABLE Employees ADD email VARCHAR(100)' highlighted in green.

```
9
10 CREATE TABLE Location (
11     location_id INT,
12     location VARCHAR(200)
13 );
14
15 CREATE TABLE Employees (
16     employee_id INT,
17     employee_name VARCHAR(50),
18     gender ENUM('M','F'),
19     age INT,
20     hire_date DATE,
21     designation VARCHAR(100),
22     department_id INT,
23     location_id INT,
24     salary DECIMAL(10,2)
25 );
26
27 ALTER TABLE Employees
28 ADD email VARCHAR(100);
```

ALTER TABLE Syntax:

`ALTER` changes the structure of a table. For example, you can add or delete columns, create or destroy indexes, change the type of existing columns, or rename columns or the table itself. You can also change characteristics such as the storage engine used for the table or the table comment.

- To use `ALTER`, you need `ALTER`, `CREATE`, and `INSERT` privileges for the table. Renaming a table requires `ALTER` and `DROP` on the old table, `ALTER`, `CREATE`, and `INSERT` on the new table.
- Following the table name, specify the alterations to be made. If none are given, `ALTER` does nothing.
- The syntax for many of the permissible alterations is similar to clauses of the `CREATE` statement. `COLUMN_DEFINITION` clauses use the same syntax for `ADD` and `CHANGE` as for `ALTER`. For more information, see [ALTER](#).
- The word `COLUMN` is optional and can be omitted, except for `RENAME COLUMN` (to distinguish a column-renaming operation from the `RENAME` table-renaming operation).
- Multiple `ADD`, `ALTER`, `DROP`, and `CHANGE` clauses are permitted in a single `ALTER` statement, separated by commas. This is a MySQL extension to standard SQL, which permits only one of each clause per `ALTER` statement. For example, to drop multiple columns in a single `ALTER` statement, use:

Action Output

#	Time	Action	Message	Duration / Fetch
1	12:51:19	USE employee		0.000 sec
3	12:52:51	CREATE TABLE Departments (department_id INT, department_name VARCHAR(100))	0 row(s) affected	0.031 sec
4	12:54:26	CREATE TABLE Location (location_id INT, location VARCHAR(200))	0 row(s) affected	0.015 sec
5	12:56:40	CREATE TABLE Employees (employee_id INT, employee_name VARCHAR(50), gender ENUM('M','F'))	0 row(s) affected	0.016 sec
6	12:59:03	ALTER TABLE Employees ADD email VARCHAR(100)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec

3.2) MODIFY THE "DESIGNATION" COLUMN TO 'VARCHAR(255)'.

The screenshot shows the same SQL IDE interface as before, but with the 'ALTER TABLE Employees MODIFY designation VARCHAR(255);' statement highlighted in the editor. The right-hand pane still shows the 'ALTER TABLE Syntax' documentation. The bottom 'Output' pane shows the execution of this statement as the final action, highlighted in green.

```
12     location VARCHAR(200)
13 );
14
15 CREATE TABLE Employees (
16     employee_id INT,
17     employee_name VARCHAR(50),
18     gender ENUM('M','F'),
19     age INT,
20     hire_date DATE,
21     designation VARCHAR(100),
22     department_id INT,
23     location_id INT,
24     salary DECIMAL(10,2)
25 );
26
27 ALTER TABLE Employees
28 ADD email VARCHAR(100);
29
30 ALTER TABLE Employees
31 MODIFY designation VARCHAR(255);
```

ALTER TABLE Syntax:

`ALTER` changes the structure of a table. For example, you can add or delete columns, create or destroy indexes, change the type of existing columns, or rename columns or the table itself. You can also change characteristics such as the storage engine used for the table or the table comment.

- To use `ALTER`, you need `ALTER`, `CREATE`, and `INSERT` privileges for the table. Renaming a table requires `ALTER` and `DROP` on the old table, `ALTER`, `CREATE`, and `INSERT` on the new table.
- Following the table name, specify the alterations to be made. If none are given, `ALTER` does nothing.
- The syntax for many of the permissible alterations is similar to clauses of the `CREATE` statement. `COLUMN_DEFINITION` clauses use the same syntax for `ADD` and `CHANGE` as for `ALTER`. For more information, see [ALTER](#).
- The word `COLUMN` is optional and can be omitted, except for `RENAME COLUMN` (to distinguish a column-renaming operation from the `RENAME` table-renaming operation).
- Multiple `ADD`, `ALTER`, `DROP`, and `CHANGE` clauses are permitted in a single `ALTER` statement, separated by commas. This is a MySQL extension to standard SQL, which permits only one of each clause per `ALTER` statement. For example, to drop multiple columns in a single `ALTER` statement, use:

Action Output

#	Time	Action	Message	Duration / Fetch
3	12:52:51	CREATE TABLE Departments (department_id INT, department_name VARCHAR(100))	0 row(s) affected	0.031 sec
4	12:54:26	CREATE TABLE Location (location_id INT, location VARCHAR(200))	0 row(s) affected	0.015 sec
5	12:56:40	CREATE TABLE Employees (employee_id INT, employee_name VARCHAR(50), gender ENUM('M','F'))	0 row(s) affected	0.016 sec
6	12:59:03	ALTER TABLE Employees ADD email VARCHAR(100)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
7	13:00:29	ALTER TABLE Employees MODIFY designation VARCHAR(255)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.016 sec

3.3) DROP THE "AGE" COLUMN FROM THE "EMPLOYEES" TABLE.

The screenshot shows a SQL IDE interface with the following components:

- Navigator:** Shows a tree view with 'schooldb' and 'sys' schemas. 'No object selected' is displayed.
- SQL Editor:** Contains the following SQL script:

```
15 CREATE TABLE Employees (  
16     employee_id INT,  
17     employee_name VARCHAR(50),  
18     gender ENUM('M','F'),  
19     age INT,  
20     hire_date DATE,  
21     designation VARCHAR(100),  
22     department_id INT,  
23     location_id INT,  
24     salary DECIMAL(10,2)  
25 );  
26  
27 ALTER TABLE Employees  
28 ADD email VARCHAR(100);  
29  
30 ALTER TABLE Employees  
31 MODIFY designation VARCHAR(255);  
32  
33 ALTER TABLE Employees  
34 DROP COLUMN age;
```

The last statement is highlighted with a green box.
SQLAdditions: A sidebar on the right titled 'ALTER TABLE Syntax' provides information about the syntax and usage of the ALTER TABLE statement.
- Output:** A table showing the execution results of the SQL statements:

#	Time	Action	Message	Duration / Fetch
4	12:54:26	CREATE TABLE Location (location_id INT, location VARCHAR(300))	0 row(s) affected	0.015 sec
5	12:56:40	CREATE TABLE Employees (employee_id INT, employee_name VARCHAR(50), gender ENUM('M','F'), ...	0 row(s) affected	0.016 sec
6	12:59:03	ALTER TABLE Employees ADD email VARCHAR(100)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
7	13:00:29	ALTER TABLE Employees MODIFY designation VARCHAR(255)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.016 sec
8	13:01:41	ALTER TABLE Employees DROP COLUMN age	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec

3.4) RENAME THE "HIRE_DATE" COLUMN TO "DATE_OF_JOINING".

The screenshot shows a SQL IDE interface with the following components:

- Navigator:** Shows a tree view with 'schooldb' and 'sys' schemas. 'No object selected' is displayed.
- SQL Editor:** Contains the following SQL script:

```
18 gender ENUM('M','F'),  
19 age INT,  
20 hire_date DATE,  
21 designation VARCHAR(100),  
22 department_id INT,  
23 location_id INT,  
24 salary DECIMAL(10,2)  
25 );  
26  
27 ALTER TABLE Employees  
28 ADD email VARCHAR(100);  
29  
30 ALTER TABLE Employees  
31 MODIFY designation VARCHAR(255);  
32  
33 ALTER TABLE Employees  
34 DROP COLUMN age;  
35  
36 ALTER TABLE Employees  
37 RENAME COLUMN hire_date TO date_of_joining;
```

The last statement is highlighted with a green box.
SQLAdditions: A sidebar on the right titled 'ALTER TABLE Syntax' provides information about the syntax and usage of the ALTER TABLE statement.
- Output:** A table showing the execution results of the SQL statements:

#	Time	Action	Message	Duration / Fetch
5	12:56:40	CREATE TABLE Employees (employee_id INT, employee_name VARCHAR(50), gender ENUM('M','F'), ...	0 row(s) affected	0.016 sec
6	12:59:03	ALTER TABLE Employees ADD email VARCHAR(100)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
7	13:00:29	ALTER TABLE Employees MODIFY designation VARCHAR(255)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.016 sec
8	13:01:41	ALTER TABLE Employees DROP COLUMN age	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
9	13:03:13	ALTER TABLE Employees RENAME COLUMN hire_date TO date_of_joining	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.015 sec

4) RENAMING TABLES:

4.1) RENAME "DEPARTMENTS" TABLE TO "DEPARTMENTS_INFO".

The screenshot shows the SQL Developer interface with the following components:

- Navigator:** Shows the 'SCHEMAS' tree with 'schooldb' and 'sys'.
- SQL Editor:** Contains the following SQL script:

```
20 hire_date DATE,  
21 designation VARCHAR(100),  
22 department_id INT,  
23 location_id INT,  
24 salary DECIMAL(10,2)  
25 );  
26  
27 ALTER TABLE Employees  
28 ADD email VARCHAR(100);  
29  
30 ALTER TABLE Employees  
31 MODIFY designation VARCHAR(255);  
32  
33 ALTER TABLE Employees  
34 DROP COLUMN age;  
35  
36 ALTER TABLE Employees  
37 RENAME COLUMN hire_date TO date_of_joining;  
38  
39 RENAME TABLE Departments TO Departments_Info;
```

The last statement is highlighted with a green box.
Output: A table showing the execution of the statements:

#	Time	Action	Message	Duration / Fetch
6	12:59:03	ALTER TABLE Employees ADD email VARCHAR(100)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
7	13:00:43	ALTER TABLE Employees MODIFY designation VARCHAR(255)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.016 sec
8	13:01:41	ALTER TABLE Employees DROP COLUMN age	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
9	13:03:13	ALTER TABLE Employees RENAME COLUMN hire_date TO date_of_joining	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.015 sec
10	13:05:16	RENAME TABLE Departments TO Departments_Info	0 row(s) affected	0.032 sec
- SQL Additions:** Contains the 'RENAME TABLE Syntax' section:

RENAME TABLE Syntax:

`RENAME` renames one or more tables. You must have `ALTER` and `DROP` privileges for the original table, and `CREATE` and `INSERT` privileges for the new table.

For example, to rename a table named `old_table` to `new_table`, use this statement:

```
>  
RENAME TABLE old_table TO new_table;
```

That statement is equivalent to the following `ALTER` statement:

```
>  
ALTER TABLE old_table RENAME new_table;
```

`RENAME TABLE`, unlike `ALTER`, can rename multiple tables within a single statement:

4.2) RENAME "LOCATION" TABLE TO "LOCATIONS".

The screenshot shows the SQL Developer interface with the following components:

- Navigator:** Shows the 'SCHEMAS' tree with 'schooldb' and 'sys'.
- SQL Editor:** Contains the following SQL script:

```
22 department_id INT,  
23 location_id INT,  
24 salary DECIMAL(10,2)  
25 );  
26  
27 ALTER TABLE Employees  
28 ADD email VARCHAR(100);  
29  
30 ALTER TABLE Employees  
31 MODIFY designation VARCHAR(255);  
32  
33 ALTER TABLE Employees  
34 DROP COLUMN age;  
35  
36 ALTER TABLE Employees  
37 RENAME COLUMN hire_date TO date_of_joining;  
38  
39 RENAME TABLE Departments TO Departments_Info;  
40  
41 RENAME TABLE Location TO Locations;
```

The last statement is highlighted with a green box.
Output: A table showing the execution of the statements:

#	Time	Action	Message	Duration / Fetch
7	13:00:29	ALTER TABLE Employees MODIFY designation VARCHAR(255)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.016 sec
8	13:01:41	ALTER TABLE Employees DROP COLUMN age	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
9	13:03:13	ALTER TABLE Employees RENAME COLUMN hire_date TO date_of_joining	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.015 sec
10	13:05:16	RENAME TABLE Departments TO Departments_Info	0 row(s) affected	0.032 sec
11	13:06:24	RENAME TABLE Location TO Locations	0 row(s) affected	0.000 sec
- SQL Additions:** Contains the 'RENAME TABLE Syntax' section (identical to the previous screenshot).

5) TRUNCATE AND DROP COMMANDS:

5.1) TRUNCATE THE "EMPLOYEES" TABLE TO REMOVE ALL ROWS.

The screenshot shows the SQL Developer interface with the 'DDL assignment - Employees D...' script. The script contains several DDL commands, and the 'TRUNCATE TABLE Employees;' command is highlighted with a green box. The output window shows the execution results, with the 'TRUNCATE TABLE Employees' command (line 12) highlighted in green. The output table is as follows:

#	Time	Action	Message	Duration / Fetch
9	13:01:41	ALTER TABLE Employees DROP COLUMN age	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
9	13:03:13	ALTER TABLE Employees RENAME COLUMN hire_date TO date_of_joining	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.015 sec
10	13:05:16	RENAME TABLE Departments TO Departments_Info	0 row(s) affected	0.032 sec
11	13:06:24	RENAME TABLE Location TO Locations	0 row(s) affected	0.000 sec
12	13:07:31	TRUNCATE TABLE Employees	0 row(s) affected	0.032 sec

5.2) DROP THE "EMPLOYEES" TABLE.

The screenshot shows the SQL Developer interface with the 'DDL assignment - Employees D...' script. The script contains several DDL commands, and the 'DROP TABLE Employees;' command is highlighted with a green box. The output window shows the execution results, with the 'DROP TABLE Employees' command (line 13) highlighted in green. The output table is as follows:

#	Time	Action	Message	Duration / Fetch
9	13:03:13	ALTER TABLE Employees RENAME COLUMN hire_date TO date_of_joining	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.015 sec
10	13:05:16	RENAME TABLE Departments TO Departments_Info	0 row(s) affected	0.032 sec
11	13:06:24	RENAME TABLE Location TO Locations	0 row(s) affected	0.000 sec
12	13:07:31	TRUNCATE TABLE Employees	0 row(s) affected	0.032 sec
13	13:08:31	DROP TABLE Employees	0 row(s) affected	0.015 sec

5.3) DROP THE "EMPLOYEE" DATABASE.

The screenshot shows a SQL IDE interface with a DDL script in the main editor and its execution output in the bottom panel. The script includes several ALTER, RENAME, TRUNCATE, and DROP statements. The final statement, `DROP DATABASE employee;`, is highlighted with a green box. An orange arrow points from this statement to the corresponding entry in the output table.

DDL assignment - Employees D.

```
28 ADD email VARCHAR(100);
29
30 ALTER TABLE Employees
31 MODIFY designation VARCHAR(255);
32
33 ALTER TABLE Employees
34 DROP COLUMN age;
35
36 ALTER TABLE Employees
37 RENAME COLUMN hire_date TO date_of_joining;
38
39 RENAME TABLE Departments TO Departments_Info;
40
41 RENAME TABLE Location TO Locations;
42
43 TRUNCATE TABLE Employees;
44
45 DROP TABLE Employees;
46
47 DROP DATABASE employee;
```

SQL Additions

DROP DATABASE Syntax:

`||` drops all tables in the database and deletes the database. Be **very** careful with this statement! To use `||`, you need the `DROP` privilege on the database. `DROP SCHEMA` is a synonym for `||`.

`IF EXISTS` is used to prevent an error from occurring if the database does not exist.

See also: [Online help drop-database](#)

Output

#	Time	Action	Message	Duration / Fetch
11	13:06:24	RENAME TABLE Location TO Locations	0 row(s) affected	0.000 sec
12	13:07:31	TRUNCATE TABLE Employees	0 row(s) affected	0.032 sec
13	13:08:31	DROP TABLE Employees	0 row(s) affected	0.015 sec
14	13:09:51	DROP DATABASE employee	2 row(s) affected	0.031 sec
15	13:15:35	DROP DATABASE employee	Error Code: 1008. Can't drop database 'employee'; database doesn't exist	0.000 sec

6) RECREATING THE DATABASE WITH CONSTRAINTS:

6.1) CREATE THE "EMPLOYEE" DATABASE AGAIN.

CREATE DATABASE Syntax:

`CREATE` creates a database with the given name. To use this statement, you need the `CREATE` privilege for the database. `CREATE SCHEMA` is a synonym for `CREATE`.

See also: [Online help create-database](#)

#	Time	Action	Message	Duration / Fetch
12	13:07:31	TRUNCATE TABLE Employees	0 row(s) affected	0.032 sec
13	13:08:31	DROP TABLE Employees	0 row(s) affected	0.015 sec
14	13:09:51	DROP DATABASE employee	2 row(s) affected	0.031 sec
15	13:15:35	DROP DATABASE employee	Error Code: 1008. Can't drop database 'employee'; database doesn't exist	0.000 sec
16	13:16:52	CREATE DATABASE employee	1 row(s) affected	0.000 sec

6.2) SELECT THE "EMPLOYEE" DATABASE.

USE Syntax:

The `USE` statement tells MySQL to use the named database as the default (current) database for subsequent statements. This statement requires some privilege for the database or some object within it.

The named database remains the default until the end of the session or another `USE` statement is issued:

```
>
USE db1;
SELECT COUNT(*) FROM mytable; # selects from db1.mytable
USE db2;
SELECT COUNT(*) FROM mytable; # selects from db2.mytable
```

The database name must be specified on a single line. Newlines in database names are not supported.

See also: [Online help use](#)

#	Time	Action	Message	Duration / Fetch
13	13:08:31	DROP TABLE Employees	0 row(s) affected	0.015 sec
14	13:09:51	DROP DATABASE employee	2 row(s) affected	0.031 sec
15	13:15:35	DROP DATABASE employee	Error Code: 1008. Can't drop database 'employee'; database doesn't exist	0.000 sec
16	13:16:52	CREATE DATABASE employee	1 row(s) affected	0.000 sec
17	13:17:46	USE employee	0 row(s) affected	0.016 sec

6.3) CREATE TABLE "DEPARTMENTS" WITH 'PRIMARY KEY', 'NOT NULL' AND 'UNIQUE' CONSTRAINTS.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following SQL script:

```
37 RENAME COLUMN hire_date TO date_of_joining;
38
39 RENAME TABLE Departments TO Departments_Info;
40
41 RENAME TABLE Location TO Locations;
42
43 TRUNCATE TABLE Employees;
44
45 DROP TABLE Employees;
46
47 DROP DATABASE employee;
48
49 CREATE DATABASE employee;
50
51 USE employee;
52
53 CREATE TABLE Departments (
54     department_id INT PRIMARY KEY,
55     department_name VARCHAR(100) NOT NULL UNIQUE
56 );
```

The SQL script is highlighted with a green box. The Output window shows the execution results:

#	Time	Action	Message	Duration / Fetch
14	13:09:51	DROP DATABASE employee	2 row(s) affected	0.031 sec
15	13:15:35	DROP DATABASE employee	Error Code: 1008. Can't drop database 'employee': database doesn't exist	0.000 sec
16	13:16:52	CREATE DATABASE employee	1 row(s) affected	0.000 sec
17	13:17:46	USE employee	0 row(s) affected	0.016 sec
18	13:19:30	CREATE TABLE Departments (department_id INT PRIMARY KEY, department_name VARCHAR(100) NOT NULL UNIQUE	0 row(s) affected	0.031 sec

6.4) CREATE TABLE "LOCATION" WITH 'PRIMARY KEY AUTO_INCREMENT', 'NOT NULL' AND 'UNIQUE' CONSTRAINTS.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following SQL script:

```
42
43 TRUNCATE TABLE Employees;
44
45 DROP TABLE Employees;
46
47 DROP DATABASE employee;
48
49 CREATE DATABASE employee;
50
51 USE employee;
52
53 CREATE TABLE Departments (
54     department_id INT PRIMARY KEY,
55     department_name VARCHAR(100) NOT NULL UNIQUE
56 );
57
58 CREATE TABLE Location (
59     location_id INT PRIMARY KEY AUTO_INCREMENT,
60     location VARCHAR(100) NOT NULL UNIQUE
61 );
```

The SQL script is highlighted with a green box. The Output window shows the execution results:

#	Time	Action	Message	Duration / Fetch
15	13:15:35	DROP DATABASE employee	Error Code: 1008. Can't drop database 'employee': database doesn't exist	0.000 sec
16	13:16:52	CREATE DATABASE employee	1 row(s) affected	0.000 sec
17	13:17:46	USE employee	0 row(s) affected	0.016 sec
18	13:19:30	CREATE TABLE Departments (department_id INT PRIMARY KEY, department_name VARCHAR(100) NOT NULL UNIQUE	0 row(s) affected	0.031 sec
19	13:22:21	CREATE TABLE Location (location_id INT PRIMARY KEY AUTO_INCREMENT, location VARCHAR(100) NOT NULL UNIQUE	0 row(s) affected	0.031 sec

6.5) CREATE TABLE "EMPLOYEES" WITH 'PRIMARY KEY', 'NOT NULL', 'CHECK', 'DEFAULT' AND 'FOREIGN KEY' CONSTRAINTS.

The screenshot displays a database IDE interface. On the left, a 'Navigator' pane shows a 'SCHEMAS' tree with 'schooldb' and 'sys' folders. The main editor window, titled 'DDL assignment - Employees D...', contains SQL code for creating a table. The code is as follows:

```
56 }
57
58 CREATE TABLE Location (
59     location_id INT PRIMARY KEY AUTO_INCREMENT,
60     location VARCHAR(100) NOT NULL UNIQUE
61 );
62
63 CREATE TABLE Employees (
64     employee_id INT PRIMARY KEY,
65     employee_name VARCHAR(50) NOT NULL,
66     gender ENUM('M', 'F'),
67     age INT CHECK (age >= 18),
68     hire_date DATE DEFAULT (CURRENT_DATE),
69     designation VARCHAR(100),
70     department_id INT,
71     location_id INT,
72     salary DECIMAL(10,2),
73     FOREIGN KEY (department_id) REFERENCES Departments(department_id),
74     FOREIGN KEY (location_id) REFERENCES Location(location_id)
75 );
```

The 'CREATE TABLE Employees' block is highlighted with a green rectangle. An orange arrow points from this block to the 'Output' pane at the bottom. The 'Output' pane shows a table of execution results:

#	Time	Action	Message	Duration / Fetch
16	13:16:54	CREATE DATABASE employee	1 row(s) affected	0.000 sec
17	13:17:46	USE employee	0 row(s) affected	0.016 sec
18	13:19:30	CREATE TABLE Departments (department_id INT PRIMARY KEY, department_name VARCHAR(100) NOT NULL, manager_id INT, FOREIGN KEY (manager_id) REFERENCES Departments(department_id))	0 row(s) affected	0.031 sec
19	13:22:21	CREATE TABLE Location (location_id INT PRIMARY KEY AUTO_INCREMENT, location VARCHAR(100) NOT NULL UNIQUE)	0 row(s) affected	0.031 sec
20	13:27:04	CREATE TABLE Employees (employee_id INT PRIMARY KEY, employee_name VARCHAR(50) NOT NULL, gender ENUM('M', 'F'), age INT CHECK (age >= 18), hire_date DATE DEFAULT (CURRENT_DATE), designation VARCHAR(100), department_id INT, location_id INT, salary DECIMAL(10,2), FOREIGN KEY (department_id) REFERENCES Departments(department_id), FOREIGN KEY (location_id) REFERENCES Location(location_id))	0 row(s) affected	0.015 sec

The last row of the output table is highlighted with a green rectangle. On the right side of the IDE, a 'SQLAdditions' pane titled 'CREATE TABLE' provides additional information about the 'CREATE TABLE' syntax, including a link to the MySQL documentation.